

DATA ANALYST
2026
SPRINT 02.T1

SILVIA VILCHES

ÍNDICE



Pág. 04 - 31



Pág. 32 - 40



Pág. 41- 46

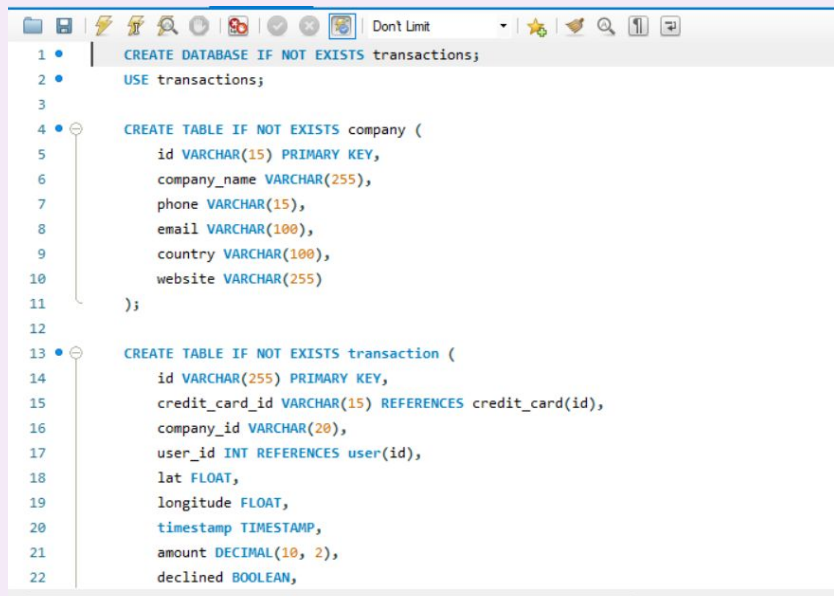


1

Ejercicio 1

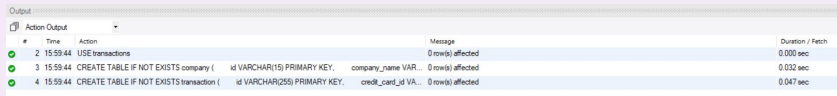
Utilizando los documentos adjuntos (estructura_dades y dades_introduir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las diferentes tablas y variables.

Abrir el documento “N1-Ex.1__estructura dades” con MySQL Workbench en el servidor local y ejecutarlo.

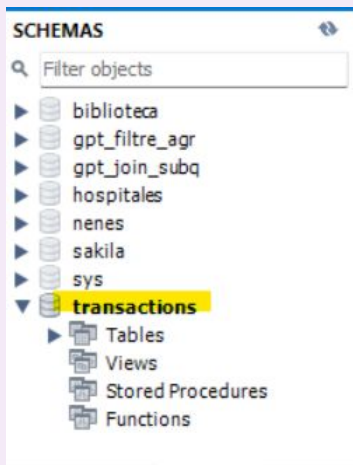


```
1 CREATE DATABASE IF NOT EXISTS transactions;
2 USE transactions;
3
4 CREATE TABLE IF NOT EXISTS company (
5     id VARCHAR(15) PRIMARY KEY,
6     company_name VARCHAR(255),
7     phone VARCHAR(15),
8     email VARCHAR(100),
9     country VARCHAR(100),
10    website VARCHAR(255)
11 );
12
13 CREATE TABLE IF NOT EXISTS transaction (
14     id VARCHAR(255) PRIMARY KEY,
15     credit_card_id VARCHAR(15) REFERENCES credit_card(id),
16     company_id VARCHAR(20),
17     user_id INT REFERENCES user(id),
18     lat FLOAT,
19     longitude FLOAT,
20     timestamp TIMESTAMP,
21     amount DECIMAL(10, 2),
22     declined BOOLEAN,
```

Una vez creado el *schema*, comprobar que se visualiza correctamente. *Refresh*



#	Time	Action	Message	Duration / Fetch
2	15:59:44	USE transactions		
3	15:59:44	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, company_name VAR...	0 row(s) affected	0.000 sec
4	15:59:44	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, credit_card_id VA...	0 row(s) affected	0.032 sec



Ejercicio 1

Utilizando los documentos adjuntos (estructura_dades y dades_introduir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las diferentes tablas y variables.

Importamos las 2 tablas: company y transaction.

Añadimos al código y ejecutamos de nuevo.

Tasca_S201_Silvia_Víches SQL File 8*

```

99909 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('086F80A1-40CA-4828-908
99910 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('E4408590-8246-47F1-B7F
99911 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('550D2A9E-8344-42D4-B48
99912 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('1269066A-DFA2-49B0-887
99913 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('2A5A5A3B-1600-4122-A1B
99914 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('7B079065-D897-4138-BB0
99915 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('6CFCE8CC-2B43-4474-942
99916 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('16DCDF4A-7A48-42BF-AD2
99917 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('A4F05A14-EFC6-4785-8C4
99918 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('CDD0FB33-F44F-43ED-8FE
99919 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('E5309321-D838-42FE-982
99920 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('BFA05A31-B8C3-4816-838
99921 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('B0EE4490-DAC2-4C18-935
99922 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('44330E06-F300-4F7D-887
99923 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('8A791E5E-859A-451F-9A7
99924 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('C30831D9-8C48-48BE-B8E
99925 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('71985C2C-4649-4A2D-B3F
99926 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('A106E074-E976-43A5-865
99927 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('9A902184-3CD5-4E5C-905
99928 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('9C401E88-FE94-44F3-878
99929 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('D162EFD4-3415-48FF-975
99930 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('5B8C65D6-71C8-42CD-AE2
  
```

Output

#	Time	Action	Message
7121	16:01:33	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) ...	1 row(s) affected
7096	16:01:33	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) ...	1 row(s) affected
4	7101	16:01:33	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) ... Running...

1

Ejercicio 1

Utilizando los documentos adjuntos (estructura_dades y dades_introduir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las diferentes tablas y variables.

Comprobamos que se visualizan correctamente los datos de ambas tablas.

company

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00043A49-2949-4946-A5D0-A5BAE3861900	C5-9294	b-2458	4713	46.1999	1.43554	2024-08-28 07:16:46	395.43	0
00444FE4830-4D3F-880E-C7B0E21CAAD	C5-5019	b-2370	438	41.5972	12.2218	2018-12-21 20:07:18	155.63	0
000452AB-E02E-4F2F-8186-CED74C975D0	C5-6699	b-2390	2118	29.7573	-95.3795	2020-07-14 15:37:45	326.01	0
000481C3-1C26-4FEF-83AD-KCD0E800488D	C5-6696	b-2230	2115	53.5489	-113.503	2017-09-04 19:44:53	161.60	0
00051AA4-9C8E-4268-8070-C38062A183E2	C5-7606	b-2266	3025	52.2084	5.69081	2017-01-05 18:19:25	148.91	0
0008A312-EDFE-4A4F-8C99-E9C92EC3CA4D	CUJ-3358	b-2598	215	53.5535	-113.499	2023-09-23 04:51:43	294.59	0
0009A151-96CF-4E31-9053-A468FF77FAA8	C5-7309	b-2546	2928	51.9362	5.34265	2023-12-31 00:06:36	383.63	0
000D0694-624D-4D99-995D-2C08A1191C7FB	C5-8483	b-2526	3962	45.492	-73.5706	2017-07-18 07:52:02	197.80	0

transaction

id	company_name	phone	email	country	website
b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.telus@yahoo.net	Germany	https://instagram.com/site
b-2226	Magna A Neque Industries	04 14 44 64 62	rius.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
b-2230	Fusce Corp.	08 14 97 58 85	rius@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2238	Convalis In Incorporated	06 66 57 29 50	mauris.ut@aol.co.uk	Germany	https://cnn.com/user/110
b-2238	Ante Jaculis Nec Foundation	08 23 04 99 53	sed.dctum.prom@outlook.ca	New Zealand	https://netflix.com/settings

1

Ejercicio 1

Utilizando los documentos adjuntos (estructura_dades y dades_introduir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las diferentes tablas y variables.

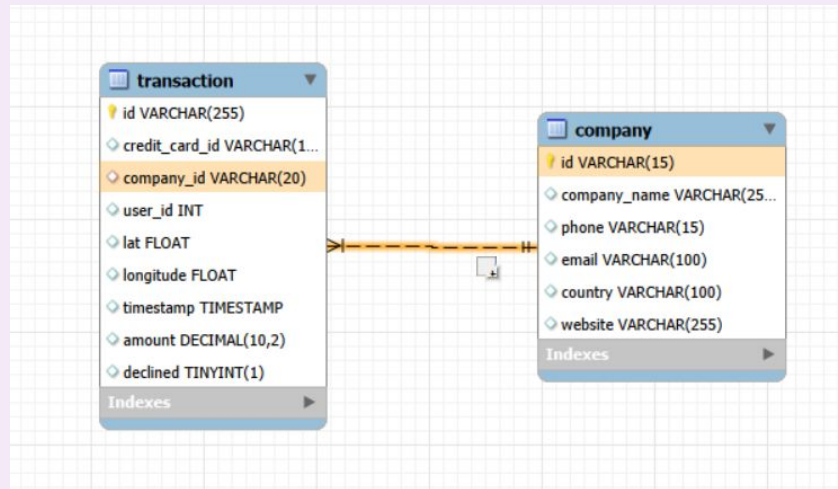
En esta base de datos tenemos 2 tablas:

- Transaction
- company

La **PRIMARY KEY** de *transaction* es id. La **FOREIGN KEY** es "company_id" que se relaciona con la **PK** de la tabla *company*, "id".

La relación es de 1:N ya que una misma compañía puede tener varias transacciones.

La tabla *transaction* es la **TABLA DE HECHOS**.



1

Ejercicio 2

Utilizando **JOIN**, realiza las siguientes consultas:

- Enumera los países que están generando ventas.

De la tabla `company` seleccionamos el campo que nos pide “países” para visualizar solamente la lista de países, sin más información.

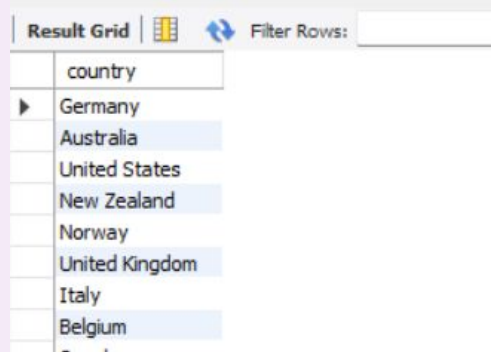
Combinamos con la tabla `transaction` a través del comando **JOIN**, donde tenemos la información de las compras.

No es necesario hacer un **SUM** ya que con **JOIN** solamente nos aparecerán los países que han realizado este tipo de operación.

Agrupamos por país para que solo aparezcan una vez.

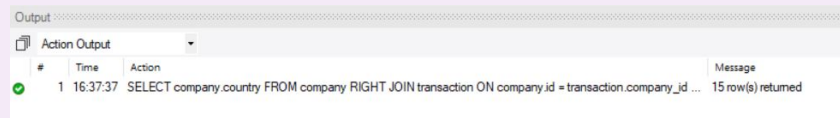
```
SELECT company.country
FROM company
JOIN transaction
ON company.id = transaction.company_id
GROUP BY company.country;
```

Nos devuelve una lista de 15 países.



The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid displays a single column named 'country' with 15 rows of data. The countries listed are Germany, Australia, United States, New Zealand, Norway, United Kingdom, Italy, and Belgium. The rows are highlighted in a light blue color.

country
Germany
Australia
United States
New Zealand
Norway
United Kingdom
Italy
Belgium



The screenshot shows the 'Output' window with the 'Action Output' tab selected. It displays a single row of information: a green checkmark, the number 1, the time 16:37:37, the SQL query text, and a message indicating that 15 rows were returned.

#	Time	Action	Message
1	16:37:37	SELECT company.country FROM company RIGHT JOIN transaction ON company.id = transaction.company_id ...	15 row(s) returned



1

Ejercicio 2

Utilizando **JOIN**, realiza las siguientes consultas:

- Desde cuántos países se generan las ventas.

Combinando las tablas *company* y *transaction*, hacemos un recuento de los distintos países que aparecen en la tabla *company* que aparecen en la tabla *transaction*.

```
11 • SELECT COUNT(distinct company.country) AS countries
12 FROM company
13 JOIN transaction
14 ON company.id = transaction.company_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [1/1](#)

countries

15

Result 8 x

Output				
Action Output				
#	Time	Action	Message	
✓ 135	12:06:05	SELECT COUNT(distinct company.country) FROM company JOIN transaction ON company.id = transaction.c...	1 row(s) returned	
✓ 136	12:06:17	SELECT COUNT(distinct company.country) AS country FROM company JOIN transaction ON company.id = tr...	1 row(s) returned	
✓ 137	12:06:22	SELECT COUNT(distinct company.country) AS countries FROM company JOIN transaction ON company.id = ...	1 row(s) returned	



1

Ejercicio 2

Utilizando **JOIN**, realiza las siguientes consultas:
- Identifica la empresa con la media de ventas más alta.

Combinamos de nuevo las tablas *company* y *transaction*.

Buscamos la media (**AVG**) de la transacciones de cada compañía agrupando por su nombre (**GROUP BY**).

Ordenamos de mayor a menor y con **LIMIT 1**, dejamos solo el primera valor, el más alto.

```
25 • SELECT company.company_name,  
26 AVG(transaction.amount) AS avg_company  
27 FROM transaction  
28 JOIN company  
29 ON company.id = transaction.company_id  
30 WHERE transaction.declined = FALSE  
31 GROUP BY company.company_name  
32 ORDER BY avg_company DESC  
33 LIMIT 1;
```

Result Grid

company_name	avg_company
Eget Ipsum Ltd	481.860000

Result 21

Output

Action Output

#	Time	Action	Message
✓ 150	12:16:20	SELECT company.company_name, AVG(transaction.amount) AS avg_company FROM transaction JOIN com...	100 row(s) returned
✓ 151	12:16:27	SELECT company.company_name, AVG(transaction.amount) AS avg_company FROM transaction JOIN com...	1 row(s) returned
✓ 152	12:16:46	SELECT company.company_name, AVG(transaction.amount) AS avg_company FROM transaction JOIN com...	1 row(s) returned



Ejercicio 3

- Muestra todas las transacciones realizadas por empresas de Alemania.

SUBCONSULTA: En la tabla de company filtramos por “country” Alemania.

```
SELECT id
FROM company
WHERE country = "Germany";
```

Nos devuelve una lista de 8 compañías que operan en Alemania.

Result Grid  Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 

id
b-2222
b-2234
b-2302
b-2306
b-2358

company 24 x

Output

Action Output 

#	Time	Action	Message
1	13:45:57	SELECT id FROM company WHERE country = "Germany"	8 row(s) returned

Aplicamos esta subconsulta como un filtro a nuestra query final donde seleccionamos las transacciones realizadas de la tabla **transaction** y filtramos por aquellas que el id de la empresa coincida con el id de nuestra subconsulta.

```
45 • SELECT id
46 FROM transaction
47 WHERE company_id IN (SELECT id
48                      FROM company
49                      WHERE country = "Germany")
50 AND declined = FALSE;
51
```

Result Grid   Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 

id
1088 ID ID-5B23-A76C-55EF-C568E49A05DD
AB069F53-965E-A2A8-CE06-CAB84FD92501
0466A42E-47CF-8D24-FD01-C0B689713128
0A476ED9-0C13-1962-F87B-D3563924B539
122DC333-E19F-D629-DCD8-9C54CF1EB89A

transaction 12 x

Output

#	Time	Action	Message
76	13:37:16	SELECT id FROM transaction WHERE company_id IN (SELECT id FROM company WHERE country = "Ger...	118 row(s) returned
77	13:37:24	SELECT id FROM transaction WHERE company_id IN (SELECT id FROM company WHERE country = "Ger...	111 row(s) returned
78	13:38:02	SELECT id FROM transaction WHERE company_id IN (SELECT id FROM company WHERE country = "Ger...	111 row(s) returned

Ésto nos devuelve 111 registros, es decir, el número de transacciones realizadas por empresas que están ubicadas en Alemania.

1

Ejercicio 3

Utilizando únicamente **subconsultas** (sin usar JOIN):

- Enumera las empresas que han realizado transacciones por un importe superior a la media de todas las transacciones.

SUBCONSULTA: En la tabla de *transaction* calculamos la media total de todas las ventas con el comando **AVG** siempre y cuando no hayan sido rechazadas.

```
12 • SELECT AVG(amount)
13 FROM transaction
14 WHERE DECLINED = FALSE;
```

Result Grid

AVG(amount)
258.715968

Result 6 x

Output

Action Output

#	Time	Action	Message
85	13:40:24	SELECT * FROM transactions.transaction WHERE DECLINED = FALSE	501 row(s) returned
86	13:41:08	SELECT company_name FROM company WHERE id IN (SELECT company_id FROM transaction WHERE a...	50 row(s) returned
87	13:42:35	SELECT AVG(amount) FROM transaction WHERE DECLINED = FALSE	1 row(s) returned

SUBCONSULTA 2: Filtramos en la tabla *transaction* las compañías que tienen transacciones superiores a la media calculada en la consulta anterior.

```
8 • SELECT company_id
9 FROM transaction
10 WHERE amount > (
11     SELECT AVG(amount)
12     FROM transaction
13     WHERE declined = FALSE)
14 GROUP BY company_id;
```

Result Grid

company_id
b-2222
b-2226
b-2230
b-2238
b-2242

transaction 9 x

Output

Action Output

#	Time	Action	Message
88	13:44:28	SELECT company_id FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction	292 row(s) returned
89	13:44:44	SELECT company_id FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction	292 row(s) returned
90	13:45:45	SELECT company_id FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction	68 row(s) returned

Nos devuelve un listado de empresas con un total de 68 registros que equivalen a las 68 transacciones que superan la media.

1

Ejercicio 3

Utilizando únicamente **subconsultas** (sin usar JOIN):

- Enumera las empresas que han realizado transacciones por un importe superior a la media de todas las transacciones.

CONSULTA FINAL: Finalmente, para obtener la lista con el nombre de las empresas y no el ID, añadimos las 2 subquers anteriores a la consulta final.

```
54 • SELECT company_name
55 FROM company
56 WHERE id IN (
57     SELECT company_id
58     FROM transaction
59     WHERE amount > (
60         SELECT AVG(amount)
61         FROM transaction
62         WHERE declined = FALSE
63     )
64 );
```

Result Grid | Filter Rows:

Exports | Wrap Cell Content:

company_name
Viverra Donec Foundation
Vestibulum Lorem PC
Ut Semper Foundation
Urna Convalis Associates
Turpis Company

company 14 x

Output

#	Time	Action	Message
89	13:44:44	SELECT company_id FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction	... 292 row(s) returned
90	13:45:45	SELECT company_id FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction	... 68 row(s) returned
91	13:46:56	SELECT company_name FROM company WHERE id IN (SELECT company_id FROM transaction WHERE a...	68 row(s) returned

1

Ejercicio 3

Utilizando únicamente **subconsultas** (sin usar JOIN):

- Eliminarán del sistema las empresas que no tienen transacciones registradas; proporciona una lista de estas empresas.

SUBCONSULTA: Seleccionamos todas las compañías que hay en la tabla de *transaction*. Utilizamos DISTINCT para evitar que haya id repetidos.

```
SELECT distinct company_id
FROM transaction;
```

Nos devuelve un listado con un total de 100 compañías.

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

company_id

b-2222

b-2226

b-2230

b-2234

b-2238

b-2242

h-2246

transaction 2

Output

Action Output

#	Time	Action	Message
710	19:53:17	SELECT company_id FROM transaction	587 row(s) returned
711	19:54:35	SELECT company_name FROM company WHERE id NOT IN (SELECT company_id FROM transaction)	0 row(s) returned
712	19:56:24	SELECT distinct company_id FROM transaction	100 row(s) returned

CONSULTA: Finalmente, filtramos por las compañías que SI están en la tabla *company* pero que NO están en la tabla *transaction*. Es decir, que no han hecho ninguna transacción o compra.

```
SELECT company_name
FROM company
WHERE id NOT IN (
    SELECT company_id
FROM transaction
);
```

Afortunadamente no nos devuelve ningún registro, es decir, no habrá que eliminar ninguna compañía del sistema.

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

company_name

company 3

Output

Action Output

#	Time	Action	Message
709	19:52:59	SELECT company_name FROM company WHERE id NOT IN (SELECT company_id FROM transaction)	0 row(s) returned
710	19:53:17	SELECT company_id FROM transaction	587 row(s) returned
711	19:54:35	SELECT company_name FROM company WHERE id NOT IN (SELECT company_id FROM transaction)	0 row(s) returned

1

Ejercicio 4

Tu tarea consiste en diseñar y crear una tabla llamada «*credit_card*» que almacene datos esenciales sobre tarjetas de crédito. La nueva tabla debe poder identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas («*transaction*» y «*company*»). Después de crear la tabla, deberás introducir la información del documento titulado «*insert_credit_data*». Recuerde mostrar el diagrama y proporcionar una breve descripción del mismo.

Creemos la tabla donde ponemos el campo ID como **PRIMARY KEY** y poder relacionarla con la tabla *transaction*.

Ponemos un número máximo de caracteres que debe tener cada registro.

```
CREATE TABLE IF NOT EXISTS credit_card (
    id VARCHAR(300) PRIMARY KEY,
    iban VARCHAR(40),
    pan VARCHAR(20),
    pin VARCHAR(10),
    cvv VARCHAR(10),
    expiring_date VARCHAR(10)
);
```

Después, introducimos los datos de la tabla que están en el archivo "*dades_introduir_credit*" y ejecutamos. La tabla se ha creado correctamente.

```
-- Insertamos datos de credit_card
1 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
2 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
3 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
4 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
5 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
6 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
7 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
8 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
9 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
10 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
11 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
12 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
13 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C
```

Result Grid						
Filter Rows:						
	id	iban	pan	pin	cvv	expiring_date
	CcS-4866	XX4792859188206596406839	8080768801072613	9271	961	02/28/25
	CcS-4867	XX6038298816319374853717	7761849537661098	4820	862	11/30/25
	CcS-4868	XX3929101617300533068044	4160419663151801	6860	549	08/25/25
	CcS-4869	XX1566712096111531886465	5892930716233310	4775	448	09/29/25
	CcS-4870	XX6443663804167732133949	2227240879956178	5872	715	01/25/25

credit_card 1 x

Output

Action Output

#	Time	Action	Message
5001	20:15:15	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX915670516405388...	1 row(s) affected
5002	20:16:27	SELECT * FROM transactions.credit_card	5000 row(s) returned

1

Ejercicio 4

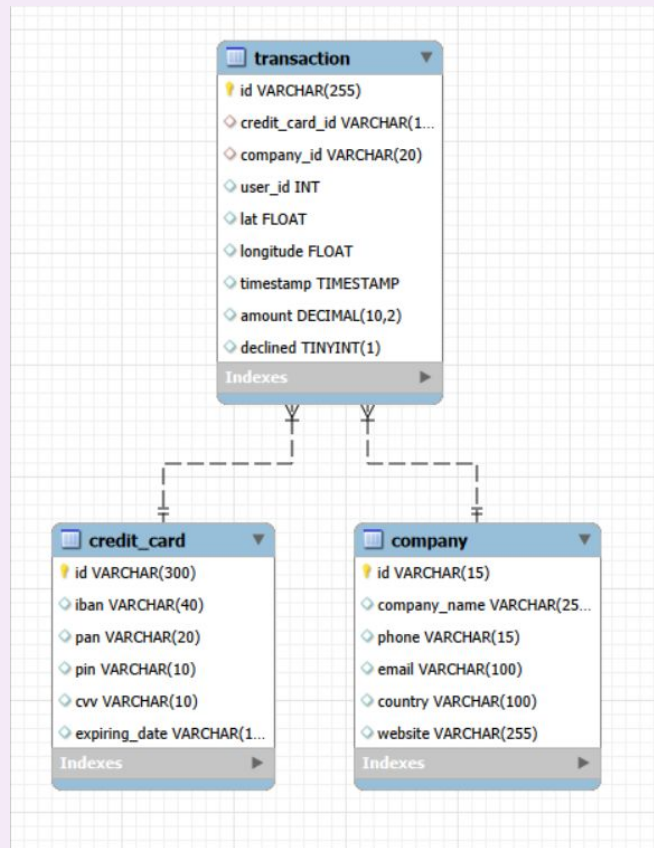
Tu tarea consiste en diseñar y crear una tabla llamada «*credit_card*» que almacene datos esenciales sobre tarjetas de crédito. La nueva tabla debe poder identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas («*transaction*» y «*company*»). Después de crear la tabla, deberás introducir la información del documento titulado «*insert_credit_data*». Recuerde mostrar el diagrama y proporcionar una breve descripción del mismo.

Debemos crear una relación entre las tablas ya existentes, como *company* o *transaction*, y la tabla nueva *credit_card*. Esto lo haremos con la instrucción **ALTER TABLE**.

Ésto nos establecer una relación sin necesidad de volver a crear todas las tablas. Establecemos que la **FOREIGN KEY** de la tabla *transaction* sea *credit_card_id*..

```
ALTER TABLE transaction
ADD CONSTRAINT fk_credit_card
FOREIGN KEY (credit_card_id)
REFERENCES credit_card(id);
```

De este modo creamos un modelo con las 3 tablas de la BBDD en el cual *transaction* es la tabla de hechos que se relaciona con el resto.



1

Ejercicio 5

El departamento de Recursos Humanos ha detectado un error en el número de cuenta asociado a la tarjeta de crédito con ID CcU-2938. La información que debe mostrarse para este registro es: TR323456312213576817699999. Recuerda indicar que se ha realizado el cambio.

Para modificar un registro utilizamos **UPDATE**. En este caso, de la tabla *credit_card* actualizaremos el campo **IBAN** donde filtraremos por el id de la tarjeta de crédito. Actualizamos el registro y ejecutamos.

```
UPDATE credit_card
SET iban = "TR323456312213576817699999"
WHERE id = "CcU-2938";
```

Vamos a comprobar que el registro se ha actualizado correctamente.

```
SELECT *
FROM credit_card
WHERE id = "CcU-2938";
```

Podemos comprobar que el tabla, el dato que nos pide el enunciado del ejercicio se ha actualizado.

Result Grid						
Filter Rows:						
Edit:						
Export/Import:						
Wra						
	id	iban	pan	pin	cvv	expiring_date
▶	CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	10/30/22
*	NULL	NULL	NULL	NULL	NULL	NULL

1

Ejercicio 6

Inserta una nueva transacción en la tabla «transaction» con la siguiente información:

Primero de todo, revisamos que no existen estos registros que nos piden. Filtro por el id de la transacción y por el id de la tarjeta de crédito. Efectivamente, ambas me devuelven 0 resultados. No existen.

```
SELECT id
FROM transaction
Where id = "10881D1D-5B23-A76C-55EF-C568E49A99DD";
```

Result Grid	
id	
HULL	

```
SELECT credit_card_id
FROM transaction
Where credit_card_id = "CcU-9999";
```

Result Grid	
credit_card_id	

Comprobado el primer paso, vamos a añadir el nuevo registro que nos pide el ejercicio con el comando INSERT TO. Especificamos la tabla donde queremos añadir el nuevo registro, los campos y los valores de cada uno.

```
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ('10881D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', '10881D1D-5B23-A76C-55EF-C568E49A99DD', '10881D1D-5B23-A76C-55EF-C568E49A99DD', 40.7128, -87.6301, 100.00, 0);
```

Output			
#	Time	Action	Message
5019	12:24:40	SELECT credit_card_id FROM transaction Where credit_card_id = "CcU-9999"	0 row(s) returned
5020	12:29:19	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ...	Error Code: 1452 Cannot add or update a child row: a foreign key constraint fails (transactions.transaction) ...

Mi primera solución fué deshabilitar todas las relaciones de la BBDD. Pero finalmente mi solución ha sido eliminar la Foreign Key para insertar el registro y crearla de nuevo más tarde. Forzar el sistema,

```
SHOW CREATE TABLE transaction;
```

La Foreign Key es "company_id".



1

Ejercicio 6

Inserta una nueva transacción en la tabla «*transaction*» con la siguiente información:

Vamos a eliminar la Foreign Key para evitar esta restricción con el comando **DROP FOREIGN KEY**.

```
ALTER TABLE transaction
DROP FOREIGN KEY company_id;
```

Antes de insertar el nuevo registro comprobamos nuestra tabla *transaction*. Nos devuelve 587 registros.

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
0668296C-CD89-A883-76BC-2E4C4F8C8AE	CcU-3743	b-2618	265	-34.3593	-100.556	2022-01-26 02:07:14	394.18	0
1017AA59-3D5F-7A4C-1992-D151A8D1FA0A	CcU-3701	b-2618	267	4.27645	-101.554	2021-11-01 01:02:11	447.11	0
18D648AB-7D54-A3E8-2D8A-2D8D2991CA43	CcU-3771	b-2618	263	-67.118	-174.141	2021-11-02 03:08:14	305.02	0
1A9B5411-64D8-DEC4-AC74-43E235AAEFC1	CcU-3687	b-2618	267	44.3314	149.168	2021-08-22 23:51:51	306.96	0
1DE927AB-29A8-654F-3372-43971B684AA	CcU-3659	b-2618	267	79.9467	90.3733	2021-08-05 15:52:25	440.13	0
1FB125A8-0901-63AC-E2B7-349E5974FBAD	CcU-3722	b-2618	267	-44.4339	-82.0844	2022-02-16 14:33:25	415.11	0
22C9EC22-45FC-72C8-63CE-5FA99A24E770	CcU-3645	b-2618	267	28.4439	-11.3959	2021-06-11 16:08:49	196.71	0

Volvemos a intentar insertar el registro nuevo,

```
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
VALUES ("18881D1D-5B23-A76C-55EF-C568E49A99D0", "CcU-9999", "b-9999", "9999", "829.999", "-117.999", "111.11", "0");
```

Comprobamos la tabla de nuevo. Ahora tenemos 588 registros.

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
02C6201E-D90A-1859-B4EE-88D2986D3802	CcU-2938	b-2362	92	81.9185	-12.5276	2021-08-28 23:42:24	466.92	0
0466A42E-47CF-8D24-FD01-C0B689713128	CcU-4219	b-2302	170	-43.9695	-117.525	2021-07-26 07:29:18	49.53	0
063FBA79-99EC-66FB-29F7-25726D1764A5	CcU-2987	b-2250	275	-81.2227	-129.05	2022-01-06 21:25:27	92.61	0
0668296C-CD89-A883-76BC-2E4C4F8C8AE	CcU-3743	b-2618	265	-34.3593	-100.556	2022-01-26 02:07:14	394.18	0
06CD9AA5-9B42-D684-DDDD-A5E394FEB499	CcU-2959	b-2346	92	33.7381	158.298	2021-10-26 23:00:01	279.93	0
07A46D48-31A3-7E87-65B9-QDA902AD109F	CcU-3225	b-2386	272	38.8342	92.1905	2021-06-28 21:11:42	340.87	1
09CE92CE-6F27-2BB7-13B5-9385B2B38E2	CcU-3071	b-2298	275	71.1706	10.5757	2021-05-11 20:40:06	303.05	1

transaction 5 x

#	Time	Action	Message
15	12:53:51	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ("10... 1 row(s) affected	
16	12:55:17	SELECT * FROM transaction	588 row(s) returned

1

Ejercicio 6

Inserta una nueva transacción en la tabla «*transaction*» con la siguiente información:

Para que localice y relacione las tablas, he creado nuevos registros en *company* y en *credit_card* para poder después establecer la **FOREIGN KEY** de nuevo que había eliminado.

```
INSERT INTO credit_card (id)
VALUES ("CcU-9999");
```

```
INSERT INTO company (id)
VALUES ("Nueva empresa");
```

Por último, eliminamos los registro sin datos que habíamos creado anteriormente en *company* y en *credit_card*.

```
DELETE FROM company WHERE id = "Nueva empresa";
DELETE FROM credit_card WHERE id = "CcU-9999";
```

✓	20	16:39:12	DELETE FROM company WHERE id = "Nueva empresa"	1 row(s) affected
✓	21	16:40:02	DELETE FROM credit_card WHERE id = "CcU-9999"	1 row(s) affected

Ahora si que podemos reasignar la **FOREING KEY** a la tabla *transaction*.

```
ALTER TABLE transactions.transaction
ADD CONSTRAINT company_id
FOREIGN KEY (company_id)
REFERENCES transactions.company (id);
```

✓	22	16:40:21	ALTER TABLE transaction ADD CONSTRAINT fk_company_id FOREIGN KEY (company_id) REFERENCES co...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
---	----	----------	--	--

1

Ejercicio 7

El departamento de Recursos Humanos te ha solicitado que elimines la columna «pan» de la tabla *credit_card*. Recuerda mostrar el cambio que se ha realizado.

Primero de todo comprobamos la tabla *credit_card* y que la columna “Pan” que nos piden eliminar existe y se llama tal y como nos lo indican.

```
115 • SELECT *
116 FROM credit_card;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

	id	iban	pan	pin	cvv	expiring_date
▶	CcS-4857	XX4857591835292505850771	2314242385113924	1819	467	09/27/25
	CcS-4858	XX8581768137002436094025	6582720299715533	3964	817	12/28/28
	CcS-4859	XX7826930491423553609370	8861684536289642	4983	277	11/26/26
	CcS-4860	XX5559590368835304645299	2481155515498459	6876	661	07/27/27
	CcS-4861	XX2035182877195191627307	1308930301149557	5710	398	04/25/26
	CcS-4862	XX4774721462463645409758	6715617009807829	4042	174	11/27/26
	CcS-4863	XX1476829664245046207111	3140879819451394	5969	449	12/27/29

credit_card 16 x

Output

Action Output

#	Time	Action
✓ 1	13:26:38	SELECT * FROM credit_card

Ejecutamos el comando eliminar columna de una tabla. Indicamos el nombre de la tabla y con el comando **DROP COLUMN** indicamos el nombre de la columna que queremos eliminar.

```
117
118 • ALTER TABLE credit_card
119 DROP COLUMN pan;
120
```

Output

Action Output

#	Time	Action
✓ 1	13:26:38	SELECT * FROM credit_card
✓ 2	13:27:14	ALTER TABLE credit_card DROP COLUMN pan



1

Ejercicio 7

El departamento de Recursos Humanos te ha solicitado que elimines la columna «*pan*» de la tabla *credit_card*. Recuerda mostrar el cambio que se ha realizado.

Por último, comprobamos que la columna ha sido eliminado correctamente.

```
121 • SELECT *
122 FROM credit_card;
```

Result Grid

	id	iban	pin	cvv	expiring_date
►	CcS-4857	XX4857591835292505850771	1819	467	09/27/25
	CcS-4858	XX8581768137002436094025	3964	817	12/28/28
	CcS-4859	XX7826930491423553609370	4983	277	11/26/26
	CcS-4860	XX5559590368835304645299	6876	661	07/27/27
	CcS-4861	XX2035182877195191627307	5710	398	04/25/26
	CcS-4862	XX4774721462463645409758	4042	174	11/27/26
	CcS-4863	XX1476829664245046207111	5969	449	12/27/29

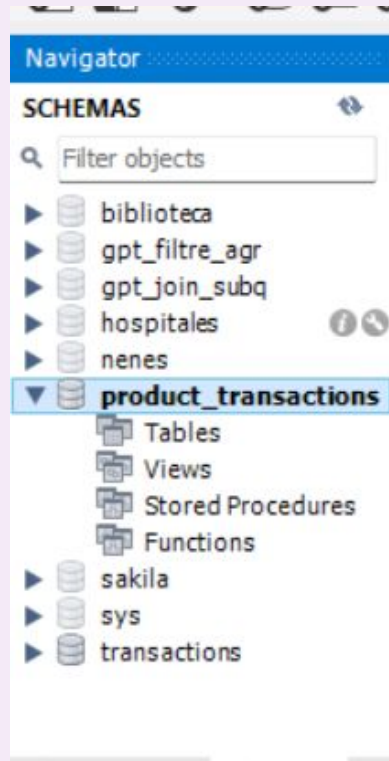
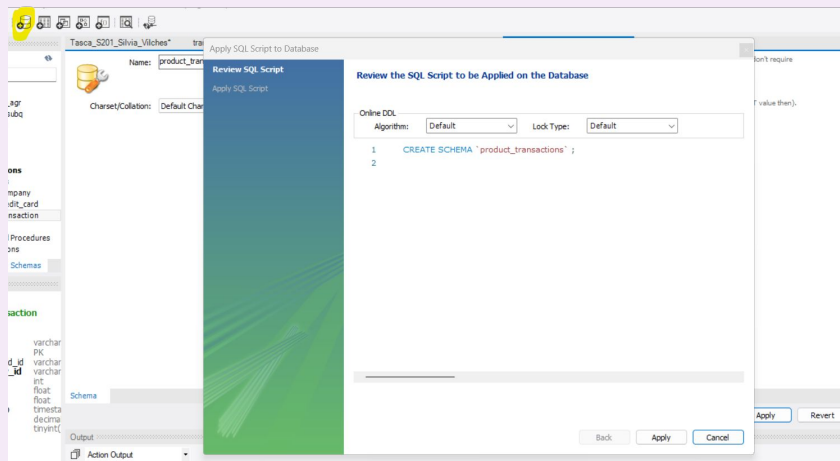
1

Ejercicio 8

Descarga los archivos CSV que encontrarás en la sección de recursos:

Estúdialos y diseña una base de datos con esquema en estrella que contenga al menos 4 tablas a partir de las cuales puedas realizar las siguientes consultas: Ejercicio 9 y 10.

Primero de todo creamos una **BBDD** donde añadiremos las tablas más tarde. Lo llamaremos *"product_transactions"*.



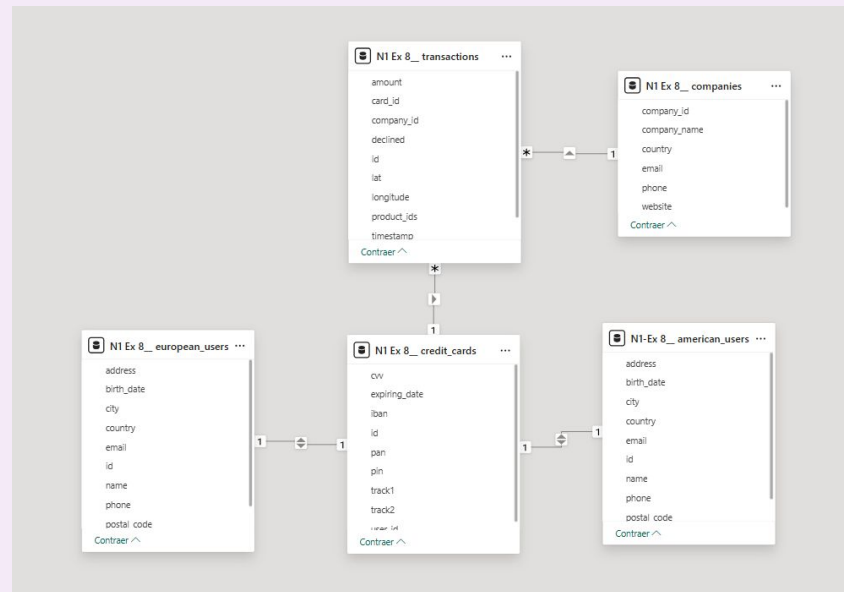
1

Ejercicio 8

Descarga los archivos CSV que encontrarás en la sección de recursos:

Estúdialos y diseña una base de datos con esquema en estrella que contenga al menos 4 tablas a partir de las cuales puedas realizar las siguientes consultas: Ejercicio 9 y 10.

Para poder leer los datos y saber interpretarlo y relacionarlos, los he abierto y creado el modelo en PowerBI.



- La tabla de hechos es *transactions* cuya PK es *Id*.
- La relacionamos con *companies* a través del campo *business_id* - *company_id*.
- A través del campo *credit_card_id* se relaciona con la tabla *credit_cards*.
- Las tablas *american_users* y *european_users* se conecta con *credit_card* en modo copo de nieve. Estas tablas podrían formar una sola si lo quisiéramos a través de los campos *user_id* - *id*.

Una vez entendidos los datos, procedemos a crear las tablas antes de cargar los archivos .csv.

CREAMOS LA TABLA *companies*

```
-- Creamos la tabla companies
CREATE TABLE IF NOT EXISTS companies (
  company_id VARCHAR(15) PRIMARY KEY,
  company_name VARCHAR(255),
  phone VARCHAR(15),
  email VARCHAR(100),
  country VARCHAR(100),
  website VARCHAR(255)
);
```

Establecemos la PRIMARY KEY y detallamos el tipo de dato que será cada campo con un número de caracteres máximo.

Output			
	Time	Action	Message
22	16:40:21	ALTER TABLE transaction ADD CONSTRAINT fk_company_id FOREIGN KEY (company_id) REFERENCES co...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
23	17:20:06	Apply changes to product_transactions	Changes applied
24	17:43:58	CREATE TABLE IF NOT EXISTS companies (company_id VARCHAR(15) PRIMARY KEY, company_n...	0 row(s) affected

1

Ejercicio 8

Descarga los archivos CSV que encontrarás en la sección de recursos:

Estúdialos y diseña una base de datos con esquema en estrella que contenga al menos 4 tablas a partir de las cuales puedas realizar las siguientes consultas: Ejercicio 9 y 10.

CREAMOS LA TABLA `credit_cards`

```

149 -- Creamos la tabla credit_cards
150 CREATE TABLE IF NOT EXISTS credit_cards (
151     id VARCHAR(15) PRIMARY KEY,
152     user_id VARCHAR(25),
153     iban VARCHAR(40),
154     pan VARCHAR(10),
155     pin VARCHAR(10),
156     cvv VARCHAR(10),
157     track1 VARCHAR(10),
158     track2 VARCHAR(10),
159     expiring_date VARCHAR(10)
160 );

```

#	Time	Action	Message
29	10:15:56	SELECT * FROM transactions.credit_card	5000 row(s) returned
30	10:16:14	show create table transactions.credit_card	1 row(s) returned
31	10:17:37	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(15) PRIMARY KEY, user_id VARCHAR(25), ...	0 row(s) affected

CREAMOS LA TABLA `american_users`

```

179 -- Creamos la tabla american_users
180 CREATE TABLE IF NOT EXISTS american_users (
181     id VARCHAR(40) PRIMARY KEY,
182     name VARCHAR(40),
183     surname VARCHAR(40),
184     phone VARCHAR(40),
185     email VARCHAR(40),
186     birth_date VARCHAR(40),
187     country VARCHAR(40),
188     city VARCHAR(40),
189     postal_code VARCHAR(40),
190     address VARCHAR(40)
191 );

```

#	Time	Action	Message
1	10:24:34	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(15) PRIMARY KEY, user_id VARCHAR(25), ...	0 row(s) affected
2	10:24:37	CREATE TABLE IF NOT EXISTS european_users (id VARCHAR(40) PRIMARY KEY, name VARCHAR(40), surnam...	0 row(s) affected
3	10:27:00	CREATE TABLE IF NOT EXISTS american_users (id VARCHAR(40) PRIMARY KEY, name VARCHAR(40), surnam...	0 row(s) affected

CREAMOS LA TABLA `european_users`

```

165 -- Creamos la tabla european_users
166 CREATE TABLE IF NOT EXISTS european_users (
167     id VARCHAR(40) PRIMARY KEY,
168     name VARCHAR(40),
169     surname VARCHAR(40),
170     phone VARCHAR(40),
171     email VARCHAR(40),
172     birth_date VARCHAR(40),
173     country VARCHAR(40),
174     city VARCHAR(40),
175     postal_code VARCHAR(40),
176     address VARCHAR(40)
177 );

```

#	Time	Action	Message
1	10:24:34	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(15) PRIMARY KEY, user_id VARCHAR(25), ...	0 row(s) affected
2	10:24:37	CREATE TABLE IF NOT EXISTS european_users (id VARCHAR(40) PRIMARY KEY, name VARCHAR(40), surnam...	0 row(s) affected

CREAMOS LA TABLA `transactions` indicando las referencias al resto de tablas.

```

-- Creamos la tabla transactions
CREATE TABLE IF NOT EXISTS transactions (
    id VARCHAR(255) PRIMARY KEY,
    credit_card_id VARCHAR(15) REFERENCES credit_cards(id),
    company_id VARCHAR(20) REFERENCES companies(company_id),
    timestamp TIMESTAMP,
    amount DECIMAL(10, 2),
    declined BOOLEAN,
    product_ids VARCHAR(20),
    user_id INT REFERENCES european_users(id),
    lat FLOAT,
    longitude FLOAT
);

```

1

Ejercicio 8

Descarga los archivos CSV que encontrarás en la sección de recursos:

Estúdialos y diseña una base de datos con esquema en estrella que contenga al menos 4 tablas a partir de las cuales puedas realizar las siguientes consultas: Ejercicio 9 y 10.

Ahora, vamos a cargar los archivos csv en cada una de las tablas que hemos creado.

```
LOAD DATA
INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__ companies.csv"
INTO TABLE companies
FIELDS TERMINATED BY ","
ENCLOSED BY '"'
IGNORE 1 ROWS;
```

Primero añadimos la ruta del archivo con LOAD DATA INFILE.

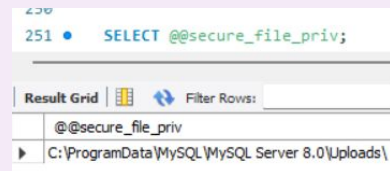
Indicamos en qué tabla irán estos datos INTO TABLE.

Después indicamos que los valores se separarán por comas y que el texto irá entre "".

Por último, le indicamos que ignore la primera fila del csv porque contiene el nombre de los campos y no datos/valores.

```
1 11:55:30 LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__ credit_cards.csv' INTO TA... Error Code: 1406. Data too long for column 'phone' at row 1
```

Tras un millón de errores, no había añadido los archivos a la ruta que me indicaba.



Una vez hecho, subimos los archivos CSV.

```
LOAD DATA
INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__ companies.csv"
INTO TABLE companies
FIELDS TERMINATED BY ","
ENCLOSED BY '"'
IGNORE 1 ROWS;

LOAD DATA
INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__ credit_cards.csv"
INTO TABLE credit_cards
FIELDS TERMINATED BY ","
ENCLOSED BY '"'
IGNORE 1 ROWS;

LOAD DATA
INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__ european_users.csv"
INTO TABLE european_users
FIELDS TERMINATED BY ","
ENCLOSED BY '"'
IGNORE 1 ROWS;
```

1

Ejercicio 8

Descarga los archivos CSV que encontrarás en la sección de recursos:

Estúdialos y diseña una base de datos con esquema en estrella que contenga al menos 4 tablas a partir de las cuales puedas realizar las siguientes consultas: Ejercicio 9 y 10.

Al cargar “transaction, de nuevo me aparecen varios errores, cambiamos el final de los datos, y sube correctamente.

```
LOAD DATA
INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__ american_users.csv"
INTO TABLE american_users
FIELDS TERMINATED BY ","
ENCLOSED BY '"'
IGNORE 1 ROWS;

LOAD DATA
INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__ transactions.csv"
INTO TABLE transactions
FIELDS TERMINATED BY ";"
ENCLOSED BY '"'
IGNORE 1 ROWS;
```

15 • SELECT * FROM product_transactions.transactions;

id	credit_card_id	company_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
00043A49-2949-494B-A5D0-85BAE3B81900	C5-9294	b-2458	2024-08-28 07:16:46	395.43	0	16/ 26/ 97/ 87	4713	46.1999	1.43554
000447FE-6650-40CF-85DE-C7ED0EE1CAAD	C5-5019	b-2370	2016-12-21 20:07:18	155.63	0	66/ 69/ 87	438	41.5972	12.2218
00045D68-ED2E-4F2F-8186-CEE074D875D0	C5-6699	b-2390	2020-07-14 15:37:45	326.01	0	30/ 11/ 16/ 81	2118	29.7573	-95.3796
000481C3-1C26-4FEF-83A0-4CD0E8004EBD	C5-6696	b-2230	2017-09-04 19:44:53	161.60	0	72	2115	53.5489	-113.503
00051AA4-9C8E-4268-6070-C38062A183E2	C5-7606	b-2266	2017-01-05 18:19:25	148.91	0	18	3025	52.2084	5.69081

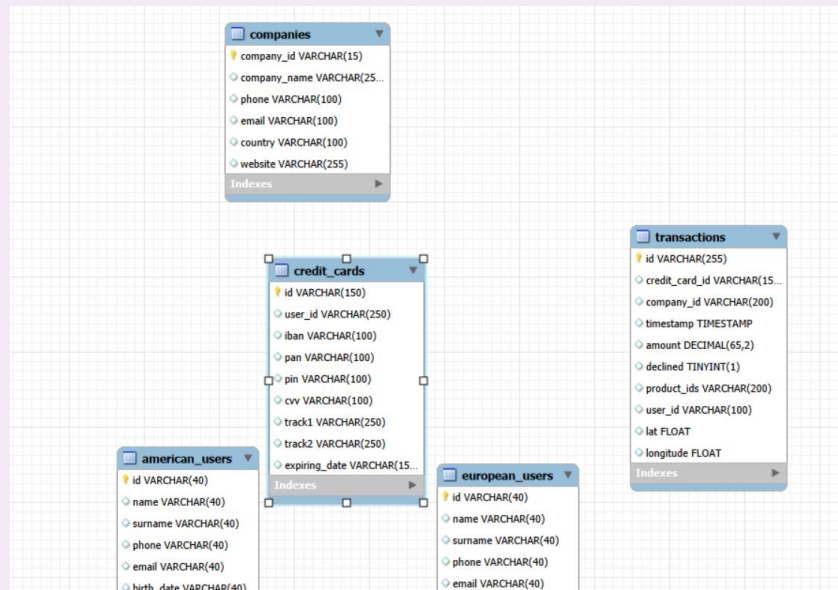
transactions 2 x

Output

#	Time	Action	Message
6	12:32:01	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(255) PRIMARY KEY, credit_card_id VARCHAR(150) FOREIGN KEY REFERENCES credit_cards(id), company_id VARCHAR(15) FOREIGN KEY REFERENCES companies(id), timestamp TIMESTAMP, amount DECIMAL(65,2), declined TINYINT(1), product_ids VARCHAR(255) FOREIGN KEY REFERENCES product_transactions.transactions(id), user_id VARCHAR(250) FOREIGN KEY REFERENCES american_users(id), lat FLOAT, longitude FLOAT)	0 row(s) affected
7	12:32:04	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__ transactions.csv' INTO TABLE transactions	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0

Vamos a crear el modelo de datos “estrella” tal y como nos indica el ejercicio Cuando creamos el esquema, aparecen todas las tablas sin conexión. Para que todo funcione correctamente, debemos relacionarlas.

Primero se crea el modelo que no aparece sin relaciones.



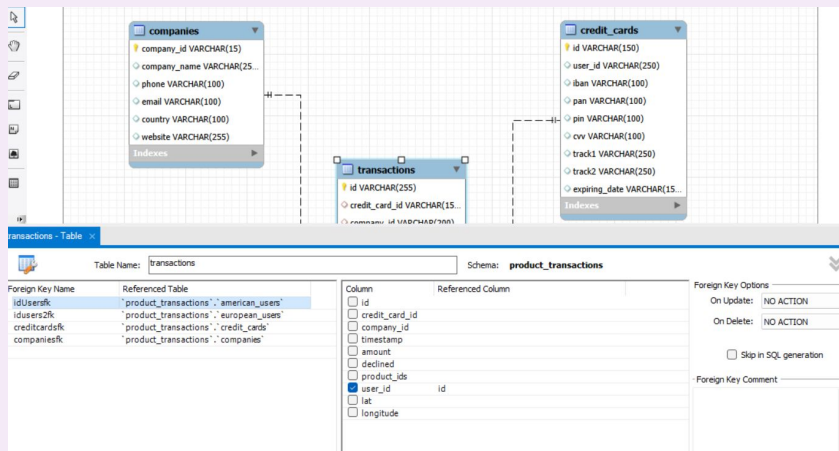
1

Ejercicio 8

Descarga los archivos CSV que encontrarás en la sección de recursos:

Estúdialos y diseña una base de datos con esquema en estrella que contenga al menos 4 tablas a partir de las cuales puedas realizar las siguientes consultas: Ejercicio 9 y 10.

Establecemos la **FOREIGN KEY** en la tabla de *transacciones* y sus relaciones con las **PRIMARY KEYS** del resto de tablas.



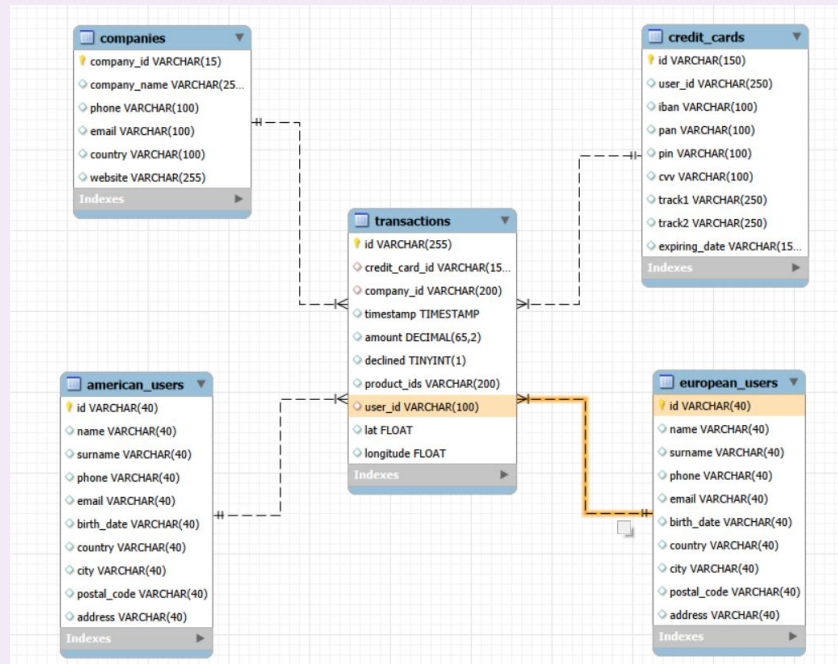
RELACIONES:

transactions → credit_cards (via credit_card_id)

transactions → companies (via company_id)

transactions → european_users (via user_id)

Tenemos un **modelo estrella** donde la *transacciones* es la tabla de hechos.



1

Ejercicio 9

Realiza una subconsulta que muestre todos los usuarios con más de 80 transacciones, utilizando al menos dos tablas.

Lo primero que haré es unir ambas tablas de usuarios en una sola, lo que creo me ayudará a obtener los resultados. Para esto, utilizo el comando UNION, ya que ambas tablas tienen las mismas columnas y el ejercicio no me dice de diferenciar los usuarios por país.

```

251
252 • SELECT * FROM european_users
253 UNION
254 SELECT * FROM american_users;
255
256

```

Result Grid

	id	name	surname	phone	email	birth_date	country	city	postal_code	address
▶	1000	Amkgrv	Qbulxbo	+48-258-9936	amkgrv.qbulxbo@example.com	May 17, 1970	Germany	Stuttgart	70173	215 Qbulxbo St
	1001	Nfvrlb	Oydaivbg	+94-121-2522	nfvrlb.oydaivbg@example.com	Mar 4, 1994	Germany	Cologne	50667	121 Oydaivbg St
	1002	Ijbfnd	Jbddzhvp	+70-120-3668	ijbfnd.jbddzhvp@example.com	Sep 27, 2001	Germany	Munich	80331	412 Jbddzhvp St
	1003	Uycdig	Sfbdymzj	+58-123-6968	uycdig.sfbdymzj@example.com	Jan 20, 1981	Germany	Stuttgart	70173	735 Sfbdymzj St
	1004	Yjqurq	Ojzvgq	+77-944-2340	yjqurq.ojzvgq@example.com	Jul 27, 1954	Germany	Munich	80331	685 Ojzvgq St
	1005	Mrlqtu	Glofegvik	+54-801-2627	mrlqtu.glofegvik@example.com	Nov 15, 1962	Portugal	Funchal	9000-001	8 Glofegvik St
	1007	Ehcrhp	Xahkzrim	+71-862-6101	ehcrhp.xahkzrim@example.com	Nov 8, 1971	Spain	Sevilla	41001	205 Xahkzrim St

Result 35 x

Output

Action Output

#	Time	Action	Message
1	13:15:19	SELECT * FROM european_users UNION SELECT * FROM american_users	5000 row(s) returned

Esto me devuelve un listado de 5000 usuarios.

En segundo lugar, crearé una subconsulta en la tabla *transactions*.

Haré un recuento de transacciones por usuarios. Para ello, utilizo el comando SUM y lo agrupo por el id de usuario (que son id únicos).

IMPORTANTE: Se ha de tener en cuenta y filtrar SOLO las transacciones que no han sido rechazadas.

```

258 • SELECT user_id,
259 COUNT(id) AS transaction_by_user
260 FROM transactions
261 WHERE declined = 0
262 GROUP BY user_id
263 HAVING transaction_by_user > 80;
264

```

Result Grid

	user_id	transaction_by_user
▶	318	86
	289	91
	185	107

Result 37 x

Output

Action Output

#	Time	Action	Message
2	13:16:43	SELECT user_id, COUNT(id) AS transaction_by_user FROM transactions GROUP BY user_id HAVING transactio...	4 row(s) returned
3	13:19:36	SELECT user_id, COUNT(id) AS transaction_by_user FROM transactions WHERE declined = 0 GROUP BY user...	3 row(s) returned

Me devuelve un listado de 3 usuarios que han hecho más de 80 transacciones.

1

Ejercicio 9

Realiza una subconsulta que muestre todos los usuarios con más de 80 transacciones, utilizando al menos dos tablas.

Ahora nos queda identificar estos usuarios con su nombre y apellidos.

Para ellos combinamos las tablas anteriores.

```
268 • SELECT global_users.id, global_users.name, global_users.surname
269 FROM ( SELECT * FROM european_users
270 UNION
271 SELECT * FROM american_users ) AS global_users
272 JOIN ( SELECT user_id,
273 COUNT(id) AS transaction_by_user
274 FROM transactions
275 WHERE declined = 0
276 GROUP BY user_id
277 HAVING transaction_by_user > 80) AS transactions_user80
278 ON global_users.id = transactions_user80.user_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	id	name	surname
▶	185	Molly	Gillam
	289	Dxwgi	Hwvru
	318	Bnyr	Astuw

Result 38 x

Output

#	Time	Action	Message
✓ 3	13:19:36	SELECT user_id, COUNT(id) AS transaction_by_user FROM transactions WHERE declined = 0 GROUP BY user...	3 row(s) returned
✓ 4	13:22:52	SELECT global_users.id, global_users.name, global_users.surname FROM (SELECT * FROM european_users UNI...	3 row(s) returned

1

Ejercicio 10

Muestra el importe medio por IBAN de las tarjetas de crédito de la empresa "Donec Ltd", utilizando al menos dos tablas.

Combinamos 3 tablas: *companies*, *transactions* y *credit_cards*.

Vamos a calcular la media con la cláusula **AVG** de las transacciones que agrupamos por iban y por id de la tarjeta.

Tenemos en cuenta que las transacciones no hayan sido rechazadas (**DECLINED** = FALSE) y filtramos por el nombre de la empresa que nos piden.

```
260 • SELECT credit_cards.iban, credit_cards.id,  
261        AVG(transactions.amount) avg_transaction  
262 FROM transactions  
263 JOIN credit_cards  
264 ON transactions.credit_card_id = credit_cards.id  
265 JOIN companies  
266 ON transactions.company_id = companies.company_id  
267 WHERE transactions.declined = FALSE  
268 AND companies.company_name = "Donec Ltd"  
269 GROUP BY credit_cards.iban, credit_cards.id;
```

Result Grid Filter Rows: | Export: | Wrap Cell Content:

iban	id	avg_transaction
SI35069513568830361	CcU-3848	467.820000
KW8958881954473634154801236186	CcU-3841	210.470000
GB30AMTO46195520479336	CcU-3827	337.820000
AZ58645632361051026278861261	CcU-3820	207.840000
MU1854487516225689318914472171	CcU-3813	437.820000

Result 6 x

Output

Action Output

#	Time	Action	Message
✓ 7	12:51:38	SELECT * FROM product_transactions.credit_cards	5000 row(s) returned
✓ 8	12:51:44	SELECT credit_cards.iban, AVG(transactions.amount) avg_transaction FROM transactions JOIN credit_card...	370 row(s) returned
✓ 9	12:51:56	SELECT credit_cards.iban, credit_cards.id, AVG(transactions.amount) avg_transaction FROM transactions J...	370 row(s) returned



2

Ejercicio 1

Identifica los cinco días en los que la empresa generó mayores ingresos por ventas. Indica la fecha de cada transacción junto con el total de ventas.

Con el comando **DATE(timestamp)** utilizamos (extraemos) solo la fecha del registro, ya que también contiene la hora. En este caso, la hora no os interesa. Luego sumamos el importe total vendido por día con el comando **SUM(amount)**.

Para tener en cuenta únicamente las transacciones que no fueron rechazadas utilizamos **WHERE declined = FALSE**.

Por último, agrupamos las ventas por día, lo ordenamos de mayor a menor y mostramos solo el **TOP5**.

```
8 • SELECT DATE(timestamp) AS top_date, # para extraer solo la fecha de cada registro y que no aparezca la hora
9     SUM(amount) AS total_amount
10  FROM transactions
11  WHERE declined = FALSE
12  GROUP BY DATE(timestamp)
13  ORDER BY total_amount DESC
14  LIMIT 5;
```

Result Grid | Filter Rows: | Exports: | Wrap Cell Contents: |

top_date	total_amount
2022-12-13	14337.44
2019-11-18	13591.32
2023-02-20	13332.59
2017-12-20	13318.43
2019-03-18	12680.95

Result 13 x

Output

Action Output

#	Time	Action	Message
1	10:42:46	SELECT DATE(timestamp) AS top_date, # para extraer solo la fecha de cada registro y que no aparezca la hora SU...	5 row(s) returned

2

Ejercicio 2

Presenta el nombre, el número de teléfono, el país, la fecha y el importe de aquellas empresas que realizaron transacciones por un valor comprendido entre 350 y 400 euros en alguna de estas fechas: 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024. Ordena los resultados en orden descendente por cantidad.

Empezaremos generando una **CONSULTA** en la tabla *transactions* ya que aquí encontramos toda la información de las transacciones peor la información detallada de cada empresa, para visualizar el número de resultados.

Seleccionamos en id de la compañía y el *amount* de cada transacción.

Filtramos con el comando **WHERE** el total de la transacciones que se encuentran entre un rango de 350 y 400.

Después, añadimos de nuevo los filtros de las fechas que nos piden con el comando **IN**, para que tenga en cuenta las diferentes fechas.

```

18
19 • SELECT DATE(transactions.timestamp) AS date_transactions, transactions.amount
20 FROM transactions
21 WHERE transactions.amount BETWEEN 350 AND 400
22 AND DATE(transactions.timestamp) IN ("2015-04-29", "2018-07-20", "2024-03-13")
23 ORDER BY transactions.amount DESC;
24

```

date_transactions	amount
2024-03-13	399.84
2018-07-20	399.51
2015-04-29	390.69
2024-03-13	388.29
2018-07-20	373.71
2015-04-29	367.62
2018-07-20	356.87
2015-04-29	356.87

Result 51 x

Output

#	Time	Action	Message
68	18:15:57	SELECT DATE(transactions.timestamp) AS date_transactions, transactions.amount FROM transactions WHERE tr...	Error Code: 1241. Operan...
69	18:16:22	SELECT DATE(transactions.timestamp) AS date_transactions, transactions.amount FROM transactions WHERE tr...	8 row(s) returned

Esto nos devuelve un listado de 8 transacciones de entre 350 y 400.

Ahora que tenemos este listado, nos queda saber el nombre, el número de teléfono y el país de estas compañías. Para ello, vamos a combinar esta información con la tabla *companies* donde aparecen los detalles.

```

19 • SELECT companies.company_name, companies.phone, companies.country, transactions.amount,
20 DATE(transactions.timestamp) AS date_transactions
21 FROM transactions
22 JOIN companies
23 ON transactions.company_id = companies.company_id
24 WHERE transactions.amount BETWEEN 350 AND 400
25 AND DATE(transactions.timestamp) IN ("2015-04-29", "2018-07-20", "2024-03-13")
26 ORDER BY transactions.amount DESC;
27

```

company_name	phone	country	amount	date_transactions
Aliquam PC	01 45 73 52 16	Germany	399.84	2024-03-13
Auctor Mauris Vel LLP	08 09 28 74 14	United States	399.51	2018-07-20
At Pede Corp.	06 14 48 33 15	Italy	390.69	2015-04-29
Aliquam PC	01 45 73 52 16	Germany	388.29	2024-03-13
Orci Adipiscing Limited	03 18 00 77 81	United Kingdom	373.71	2018-07-20
Fringilla LLC	08 29 15 93 57	New Zealand	367.62	2015-04-29
Pede Cum Ltd	07 62 26 48 38	Norway	356.87	2018-07-20
Pede Cum Ltd	07 62 26 48 38	Norway	356.87	2018-07-20

Result 52 x

Output

#	Time	Action	Message
69	18:16:22	SELECT DATE(transactions.timestamp) AS date_transactions, transactions.amount FROM transactions WHERE tr...	8 row(s) returned
70	18:17:15	SELECT companies.company_name, companies.phone, companies.country, transactions.amount, DATE(transact...	8 row(s) returned

2

Ejercicio 3

Necesitamos optimizar la asignación de recursos, lo cual dependerá de la capacidad operativa requerida. Por lo tanto, te solicitamos información sobre el número de transacciones realizadas por las empresas. Sin embargo, el departamento de Recursos Humanos está siendo exigente y quiere una lista de las empresas en la que se especifique si tienen 400 o más transacciones, o menos.

Primero compruebo las compañías que tienen más de 400 transacciones. Para ello, hago un recuento del número de transacciones únicas a través de su ID.

```

35 • SELECT company_id,
36     COUNT(id) AS num_transactions
37 FROM transactions
38 GROUP BY company_id
39 HAVING num_transactions <= 400;
40

```

company_id	num_transactions
b-2470	399
b-2362	380
b-2286	397

Result 59 x

Output

Action Output

#	Time	Action	Message
84	18:29:02	SELECT companies.company_id, companies.company_name, COUNT(transactions.id) AS num_transactions, CAS...	100 row(s) returned
85	18:29:11	SELECT company_id, COUNT(id) AS num_transactions FROM transactions GROUP BY company_id HAVING nu...	3 row(s) returned

Solo hay 3 empresas con más de 400 transacciones.

Hago el mismo filtro para comprobar el número de empresas con menos de 400 transacciones.

```

42 • SELECT company_id,
43     COUNT(id) AS num_transactions
44 FROM transactions
45 GROUP BY company_id
46 HAVING num_transactions >= 400;
47

```

company_id	num_transactions
b-2458	1527
b-2370	1479
b-2390	424
b-2230	447
b-2266	1563
b-2598	426
b-2546	1466
...	...

Result 60 x

Output

Action Output

#	Time	Action	Message
85	18:29:11	SELECT company_id, COUNT(id) AS num_transactions FROM transactions GROUP BY company_id HAVING nu...	3 row(s) returned
86	18:32:26	SELECT company_id, COUNT(id) AS num_transactions FROM transactions GROUP BY company_id HAVING nu...	97 row(s) returned

Me devuelve un listado de 97 empresas con 400 transacciones.

2

Ejercicio 3

Necesitamos optimizar la asignación de recursos, lo cual dependerá de la capacidad operativa requerida. Por lo tanto, te solicitamos información sobre el número de transacciones realizadas por las empresas. Sin embargo, el departamento de Recursos Humanos está siendo exigente y quiere una lista de las empresas en la que se especifique si tienen 400 o más transacciones, o menos.

Como el departamento de RRHH nos pide una clasificación, añadiremos una nueva columna indicando estas 2 condiciones. Si tiene igual o más de 400 transacciones, o menos de 400 transacciones.

Utilizamos el comando **CASE** que sirve para declarar condicionales. Solo utilizamos el filtro de $\geq$ o más de 400. En el caso de que no se cumpla, nos devolverá la condición **ELSE**.

```
59 SELECT companies.company_id, companies.company_name,
60 COUNT(transactions.id) AS num_transactions,
61 CASE WHEN COUNT(transactions.id) >= 400 THEN "More than 400"
62 ELSE "400 or less"
63 END AS transaction_filter
64 FROM companies
65 LEFT JOIN transactions
66 ON companies.company_id = transactions.company_id
67 GROUP BY companies.company_id,
68 companies.company_name;
69
```

company_id	company_name	num_transactions	transaction_filter
b-2222	Ac Fermentum Incorporated	2401	More than 400
b-2226	Magna A Neque Industries	410	More than 400
b-2230	Fusce Corp.	447	More than 400
b-2234	Convallis In Incorporated	1514	More than 400
b-2238	Ante Iaculis Nec Foundation	472	More than 400
b-2242	Donec Ltd	449	More than 400
b-2246	Sed Nunc Ltd	1541	More than 400

Result 75 x

Output

Action Output

#	Time	Action	Message
127	19:59:16	SELECT companies.company_id, companies.company_name, COUNT(transactions.id) AS num_transactions, CAS...	100 row(s) returned
128	19:59:24	SELECT companies.company_id, companies.company_name, COUNT(transactions.id) AS num_transactions, CAS...	100 row(s) returned

Nos devuelve una tabla con 100 transacciones.

2

Ejercicio 4

Elimina el registro con el ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la tabla de transacciones de la base de datos.

Compruebo el número de registro de la tabla *transactions*. 100.000 resultados.

```
14 SELECT * FROM product_transactions.transactions;
```

id	credit_card_id	company_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
00043A49-2949-494B-A5D0-A5BAE38B190D	Cs5-9294	b-2458	2024-08-28 07:16:46	395.43	0	16, 26, 97, 87	4713	46.1999	1.43554
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	Cs5-5019	b-2370	2016-12-21 20:07:18	155.63	0	66, 69, 87	438	41.5972	12.2218
00045D68-ED2E-4FEF-8186-CE074D875D0	Cs5-6699	b-2390	2020-07-14 15:37:45	326.01	0	30, 11, 16, 81	2118	29.7573	-95.3796
000481C3-1C26-4FEF-83A0-4C0EB004EBD	Cs5-6696	b-2230	2017-09-04 19:44:53	161.60	0	72	2115	53.5489	-113.503
00051AA4-9CBE-4268-8070-C38062A1B3E2	Cs5-7606	b-2266	2017-01-05 18:19:25	148.91	0	18	3025	52.2084	5.69081
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CdJ-3358	b-2598	2023-09-23 04:51:43	294.59	0	35, 33, 19	215	53.5535	-113.499
0009A151-9BCF-4E31-9053-A468FF77FAAB	Cs5-7509	b-2546	2023-12-31 00:06:36	383.63	0	93, 55, 28, 91	2928	51.9362	5.34265

transactions 6 x

Output

Action Output

#	Time	Action	Message
88	18:39:07	SELECT companies.company_id, companies.company_name, COUNT(transactions.id) AS num_transactions, CAS...	100 row(s) returned
89	18:44:40	SELECT * FROM product_transactions.transactions	100000 row(s) returned

Ahora compruebo que esta transacción existe, es correcto.

```
14 SELECT *
15 FROM product_transactions.transactions
16 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
```

id	credit_card_id	company_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	Cs5-5019	b-2370	2016-12-21 20:07:18	155.63	0	66, 69, 87	438	41.5972	12.2218

transactions 7 x

Output

Action Output

#	Time	Action	Message
89	18:44:40	SELECT * FROM product_transactions.transactions	100000 row(s) returned
90	18:47:02	SELECT * FROM product_transactions.transactions WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	1 row(s) returned

Ahora, con el comando **DELETE**, borramos el registro filtrando por su valor.

```
49 DELETE FROM transactions
50 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
```

Output

Action Output

#	Time	Action	Message
90	18:47:02	SELECT * FROM product_transactions.transactions WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	1 row(s) returned
91	18:48:29	DELETE FROM transactions WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	1 row(s) affected

Comprobamos de nuevo el número de registros y nos devuelve 1 menos.

```
14 SELECT *
15 FROM product_transactions.transactions;
```

id	credit_card_id	company_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
00043A49-2949-494B-A5D0-A5BAE38B190D	Cs5-9294	b-2458	2024-08-28 07:16:46	395.43	0	16, 26, 97, 87	4713	46.1999	1.43554
00045D68-ED2E-4FEF-8186-CE074D875D0	Cs5-6699	b-2390	2020-07-14 15:37:45	326.01	0	30, 11, 16, 81	2118	29.7573	-95.3796
000481C3-1C26-4FEF-83A0-4C0EB004EBD	Cs5-6696	b-2230	2017-09-04 19:44:53	161.60	0	72	2115	53.5489	-113.503
00051AA4-9CBE-4268-8070-C38062A1B3E2	Cs5-7606	b-2266	2017-01-05 18:19:25	148.91	0	18	3025	52.2084	5.69081
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CdJ-3358	b-2598	2023-09-23 04:51:43	294.59	0	35, 33, 19	215	53.5535	-113.499
0009A151-9BCF-4E31-9053-A468FF77FAAB	Cs5-7509	b-2546	2023-12-31 00:06:36	383.63	0	93, 55, 28, 91	2928	51.9362	5.34265
0009D494-6245-4DF9-955D-2C084191CFFB	Cs5-8483	b-2526	2017-07-18 07:52:02	197.80	0	55, 8, 72	3902	45.492	-73.5706

transactions 8 x

Output

Action Output

#	Time	Action	Message
91	18:48:29	DELETE FROM transactions WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	1 row(s) affected
92	18:49:25	SELECT * FROM product_transactions.transactions	99999 row(s) returned

2

Ejercicio 5

El departamento de marketing desea acceder a información específica para llevar a cabo análisis y estrategias eficaces. Se ha solicitado una vista que proporcione datos clave sobre las empresas y sus transacciones. Deberás crear una vista denominada «MarketingView» que contenga la siguiente información: Nombre de la empresa. Teléfono de contacto. País de residencia. Compra media realizada por cada empresa. Presenta la vista creada, ordenando los datos de mayor a menor según la compra media.

Primero haremos una **SUBCONSULTA** en la tabla *transactions* que nos servirá como base para más adelante localizar los datos exactos que nos piden: nombre teléfono y país.

Calculamos la media del importe de todas las transacciones que ha hecho cada compañía. Utilizamos **AVG** para calcular la media y **GROUP BY** para agrupar por el id de la compañía. A esta nueva columna con la media calculada le he llamado *avg_company*.

Por último, ordenamos estos importes (la media por compañía) de mayor a menor con **ORDER BY** nuestra columna de media **DESC**.

```
59 • SELECT transactions.company_id,
60     AVG(transactions.amount) AS avg_company
61     FROM transactions
62     GROUP BY transactions.company_id
63     ORDER BY avg_company DESC;
64
```

Result Grid

company_id	avg_company
b-2222	284.867160
b-2282	276.158330
b-2422	274.235011
b-2538	272.214870
b-2498	270.080965
b-2570	269.599181
b-2470	268.604837

Result 65 x

Output

Action Output

#	Time	Action	Message
94	18:52:57	SELECT transactions.company_id, AVG(transactions.amount) AS avg_company FROM transactions GROUP BY tr...	100 row(s) returned
95	18:53:32	SELECT transactions.company_id, AVG(transactions.amount) AS avg_company FROM transactions GROUP BY tr...	100 row(s) returned

Para obtener una tabla con los datos exactos que el departamento de marketing nos pide, combinamos esta subconsulta de la tabla *transactions* con la tabla *companies* que contiene estos datos más concretos.

```
67 • SELECT companies.company_name, companies.phone, companies.country,
68     avg_table.avg_company
69     FROM companies
70     LEFT JOIN ( SELECT transactions.company_id,
71                 AVG(transactions.amount) AS avg_company
72                 FROM transactions
73                 GROUP BY transactions.company_id
74                 ORDER BY avg_company DESC) AS avg_table
75     ON companies.company_id = avg_table.company_id;
```

Result Grid

company_name	phone	country	avg_company
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.867160
Magna A Neque Industries	04 14 44 64 62	Australia	260.038610
Fusce Corp.	08 14 97 58 85	United States	265.228367
Convallis In Incorporated	06 66 57 29 50	Germany	257.745376
Ante Iaculis Nec Foundation	08 23 04 99 53	New Zealand	260.336419
Donec Ltd	01 25 51 37 37	Norway	265.012027
Sed Nunc Ltd	02 62 64 73 48	United Kingdom	254.133666

Result 69 x

Output

Action Output

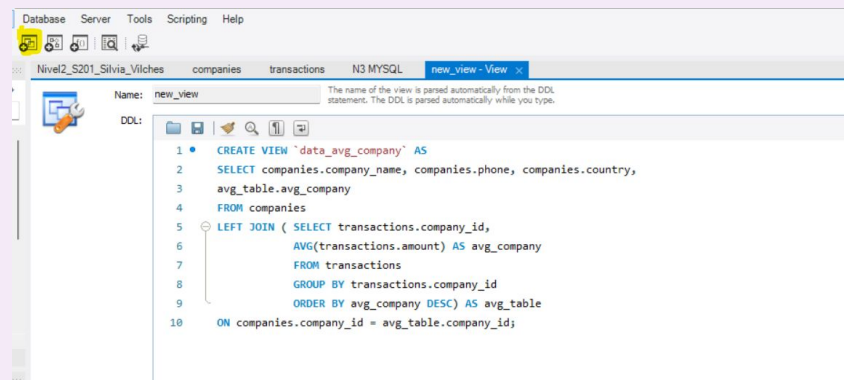
#	Time	Action	Message
98	19:05:28	SELECT companies company_name, companies.phone, companies.country, avg_table.avg_company FROM com...	100 row(s) returned
99	19:05:40	SELECT companies company_name, companies.phone, companies.country, avg_table.avg_company FROM com...	100 row(s) returned

2

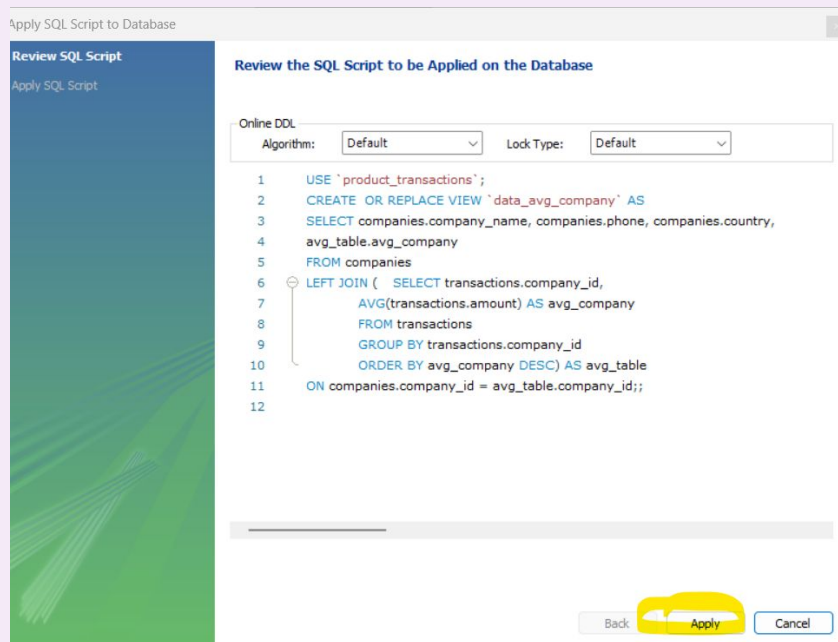
Ejercicio 5

El departamento de marketing desea acceder a información específica para llevar a cabo análisis y estrategias eficaces. Se ha solicitado una vista que proporcione datos clave sobre las empresas y sus transacciones. Deberás crear una vista denominada «MarketingView» que contenga la siguiente información: Nombre de la empresa. Teléfono de contacto. País de residencia. Compra media realizada por cada empresa. Presenta la vista creada, ordenando los datos de mayor a menor según la compra media.

Una vista ayudará al dpto. De marketing a encontrar la información de una manera rápida y evitar repetir el código y la consulta cada vez que necesiten la información. En la barra superior elegimos “Create new view” y copiamos el código de nuestra consulta ya creada.



Aplicamos.



Ejercicio 5

El departamento de marketing desea acceder a información específica para llevar a cabo análisis y estrategias eficaces. Se ha solicitado una vista que proporcione datos clave sobre las empresas y sus transacciones. Deberás crear una vista denominada «MarketingView» que contenga la siguiente información: Nombre de la empresa. Teléfono de contacto. País de residencia. Compra media realizada por cada empresa. Presenta la vista creada, ordenando los datos de mayor a menor según la compra media.

Si comprobamos en nuestra BBDD, ya tenemos la vista creada y podemos consultarla cuando queramos. Los datos se irán actualizando.

The screenshot displays a database management interface. On the left, a 'SCHEMAS' pane shows a tree view with 'product_transactions' expanded, revealing 'Tables' and 'Views'. The 'Views' section contains 'data_avg_company', which is highlighted. The main area shows a SQL query: `SELECT * FROM product_transactions.data_avg_company;`. Below the query, a 'Result Grid' displays the data. The grid has four columns: 'company_name', 'phone', 'country', and 'avg_company'. It lists eight companies with their respective phone numbers and countries, ordered by 'avg_company' in descending order. At the bottom, an 'Output' pane shows 'Action Output' with two entries: a successful update at 19:11:45 and a successful query at 19:12:40 returning 100 rows.

company_name	phone	country	avg_company
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.867160
Magna A Neque Industries	04 14 44 64 62	Australia	260.038610
Fusce Corp.	08 14 97 58 85	United States	265.228367
Convallis In Incorporated	06 66 57 29 50	Germany	257.745376
Ante Iaculis Nec Foundation	08 23 04 99 53	New Zealand	260.336419
Donec Ltd	01 25 51 37 37	Norway	265.012027
Sed Nunc Ltd	02 62 64 73 48	United Kingdom	254.133666

#	Time	Action	Message
✓ 100	19:11:45	Apply changes to data_avg_company	Changes applied
✓ 101	19:12:40	SELECT * FROM product_transactions.data_avg_company	100 row(s) returned



3

Ejercicio 1

Crea una nueva tabla que refleje el estado de las tarjetas de crédito en función de si las tres últimas transacciones han sido rechazadas; si es así, la tarjeta está inactiva; en caso contrario, está activa. A partir de esta tabla, responde a lo siguiente:

👉 ¿Cuántas tarjetas están activas?

Antes de crear la tabla, vamos a generar la consulta y comprobar los resultados. Aplicando un **CASE** condicional, sumamos el número de transacciones rechazadas. Si el resultado es mayor que 3, aparecerá “inactive” en la nueva columna. Agrupamos por *credit_card_id*.

```
7 • SELECT credit_cards.id AS credit_card_id,
8 CASE
9 WHEN SUM(transactions.declined) = 3 THEN 'inactive'
10 ELSE 'active'
11 END AS status
12 FROM credit_cards
13 JOIN transactions
14 ON credit_cards.id = transactions.credit_card_id
15 GROUP BY credit_cards.id;
```

Result Grid | Filter Rows: | Exports: | Wrap Cell Contents: | Fetch rows:

credit_card_id	status
CcS-9294	active
CcS-6699	active
CcS-6696	active
CcS-7606	active
CcU-3358	active
CcS-7509	active
CcS-8483	active

Result 72 x

Output

#	Time	Action	Message
108	19:22:36	SELECT credit_cards.id AS credit_card_id, CASE WHEN SUM(transactions.declined) = 3 THEN 'inactive' ELSE 'active' ...	5000 row(s) returned
109	19:26:44	SELECT credit_cards.id AS credit_card_id, CASE WHEN SUM(transactions.declined) = 3 THEN 'inactive' ELSE 'active' ...	5000 row(s) returned

Pero hay un problema, es que suma todas las transacciones por tarjeta, no las 3 últimas.

La función **ROW_NUMBER** añade un número ordenado de 1 a X a cada fila/registro. Así podemos clasificar las transacciones.

OVER define el conjunto de filas que más tarde con **ORDER BY**, serán ordenada según la columna que especificamos. En este caso, por fecha de más reciente a más antigua.

```
23 • SELECT credit_card_id, DATE(timestamp) AS date, declined,
24 ROW_NUMBER() OVER(PARTITION BY credit_card_id ORDER BY DATE(timestamp) DESC) AS credit_card_rank
25 FROM transactions;
26
```

Result Grid | Filter Rows: | Exports: | Wrap Cell Contents: | Fetch rows:

credit_card_id	date	declined	credit_card_rank
CcS-4857	2018-03-02	0	19
CcS-4857	2017-12-29	0	20
CcS-4857	2016-11-14	0	21
CcS-4857	2016-10-12	0	22
CcS-4858	2024-08-28	0	1
CcS-4858	2023-12-10	0	2

Result 25 x

Output

#	Time	Action	Message
41	11:48:24	SELECT credit_card_id, DATE(timestamp) AS date, declined, ROW_NUMBER() OVER(PARTITION BY credit...	99999 row(s) returned
42	11:48:27	SELECT credit_card_id, DATE(timestamp) AS date, declined, ROW_NUMBER() OVER(PARTITION BY credit...	99999 row(s) returned
43	11:48:39	SELECT credit_card_id, DATE(timestamp) AS date, declined, ROW_NUMBER() OVER(PARTITION BY credit...	99999 row(s) returned

3

Ejercicio 1

Crea una nueva tabla que refleje el estado de las tarjetas de crédito en función de si las tres últimas transacciones han sido rechazadas; si es así, la tarjeta está inactiva; en caso contrario, está activa. A partir de esta tabla, responde a lo siguiente:

👉 ¿Cuántas tarjetas están activas?

Vamos a agrupar las transacciones por tarjeta y las enumeramos, así podemos identificar las transacciones que tiene cada una de los *credit_card_id*. Las ordenamos por fechas, de más reciente a más antigua. Mi idea era poner un **LIMIT** para seleccionar solo las 3 últimas (las 3 primeras en orden descendente) pero en una función de ventana no es posible utilizar **LIMIT**. Así que utilizamos la cláusula **AS** para ponerle un nombre a esta columna (*credit_card_rank*) que más tarde utilizaremos.

```
23 • SELECT credit_card_id, DATE(timestamp) AS date, declined,
24 ROW_NUMBER() OVER(PARTITION BY credit_card_id ORDER BY DATE(timestamp) DESC) AS credit_card_rank
25 FROM transactions;
26
```

Result Grid | Filter Rows: | Exports: | Wrap Cell Contents: | Fetch rows:

credit_card_id	date	declined	credit_card_rank
CcS-4857	2018-03-02	0	19
CcS-4857	2017-12-29	0	20
CcS-4857	2016-11-14	0	21
CcS-4857	2016-10-12	0	22
CcS-4858	2024-08-28	0	1
CcS-4858	2023-12-10	0	2

Result 25 x

Action Output

#	Time	Action	Message
41	11:48:24	SELECT credit_card_id, DATE(timestamp) AS date, declined, ROW_NUMBER() OVER(PARTITION BY credit...	99999 row(s) returned
42	11:48:27	SELECT credit_card_id, DATE(timestamp) AS date, declined, ROW_NUMBER() OVER(PARTITION BY credit...	99999 row(s) returned
43	11:48:39	SELECT credit_card_id, DATE(timestamp) AS date, declined, ROW_NUMBER() OVER(PARTITION BY credit...	99999 row(s) returned

Podemos ver donde señala la flecha que la cuenta empieza de nuevo en 1 cuando cambia el id de la tarjeta.

Para poder utilizar esta consulta de manera recurrente, creamos un **CTE** (common table expression) que nos ayudará a añadirlo a una consulta de manera eficaz y dinámica.

Para definir la **CTE**, utilizamos la cláusula **WITH nombreconsulta AS** seguido del código de la consulta. A nuestra consulta CTE le llamaremos *cc_ranked*.

```
WITH cc_ranked AS (
    SELECT credit_card_id, DATE(timestamp) AS date, declined,
    ROW_NUMBER() OVER(PARTITION BY credit_card_id ORDER BY DATE(timestamp) DESC) AS credit_card_rank
    FROM transactions
)
```

Ahora, creamos una tabla como nos piden con la cláusula **CREATE TABLE**.

product_transactions

Tables

- american_users
- companies
- credit_cards
- european_users
- products
- transactions

Views

- data_avg_company

Administration Schemas

Information

Schema: product_transactions

```
27 • CREATE TABLE credit_card_status AS
28 WITH transactions_ranked AS (
29     SELECT credit_card_id, declined,
30     ROW_NUMBER() OVER(PARTITION BY credit_card_id ORDER BY DATE(timestamp) DESC) AS credit_card_rank
31     FROM transactions
32 )
33 SELECT credit_card_id,
34 CASE
35     WHEN SUM(declined) = 3 THEN 'inactive'
36     ELSE 'active'
37 END AS status
38 FROM transactions_ranked
39 GROUP BY credit_card_id;
```

3

Ejercicio 1

Crea una nueva tabla que refleje el estado de las tarjetas de crédito en función de si las tres últimas transacciones han sido rechazadas; si es así, la tarjeta está inactiva; en caso contrario, está activa. A partir de esta tabla, responde a lo siguiente:

👉 ¿Cuántas tarjetas están activas?

👉 ¿Cuántas tarjetas están activas?

Hacemos un recuento de número de “active” de la nueva tabla. No devuelve un resultado de **4990 tarjetas de crédito activas**.

```
34 • SELECT COUNT(status) as active_cc
35 FROM credit_card_status
36 WHERE status = "active";
37
38
>n
```

Result Grid

active_cc
4990

Result 33 x

Output

#	Time	Action	Message
58	12:09:29	CREATE TABLE credit_card_status AS WITH transactions_ranked AS (SELECT credit_card_id, declined, R...	5000 row(s) affected Records: 5000
59	12:11:45	SELECT COUNT(status) FROM credit_card_status WHERE status = "active"	1 row(s) returned
60	12:11:54	SELECT COUNT(status) as active_cc FROM credit_card_status WHERE status = "active"	1 row(s) returned

3

Ejercicio 2

Crea una tabla para unir los datos del archivo new products.csv con la base de datos creada, teniendo en cuenta que dispones de los product_ids de la tabla de transacciones. Genera la siguiente consulta:

👉 Necesitamos saber el número de veces que se ha vendido cada producto.

Lo primero de todo, vamos a crear la nueva tabla de *product*. Identificamos la **PRIMARY KEY** como el id del producto. Y el resto de columnas que contiene la tabla.

```
39 CREATE TABLE IF NOT EXISTS products (
40   id INT PRIMARY KEY,
41   product_name VARCHAR(255),
42   price DECIMAL(10,2),
43   colour VARCHAR(50),
44   weight DECIMAL(10,2),
45   warehouse_id VARCHAR(50)
46 );
47
```

Output

#	Time	Action	Message
119	19:33:27	SELECT credit_card_id, DATE(timestamp), CASE WHEN SUM(transactions.declined) = 3 THEN 'inactive' ELSE '...	90676 row(s) returned
120	19:52:03	CREATE TABLE IF NOT EXISTS products (id INT PRIMARY KEY, product_name VARCHAR(255), price ...	0 row(s) affected

Cargamos los datos en la tabla con la cláusula **LOAD DATA** como en anteriores ejercicios. Da error porque tenemos un símbolo en un campo **DECIMAL**.

```
50 LOAD DATA
51 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N3.Ex.2__products.csv"
52 INTO TABLE products
53 FIELDS TERMINATED BY ","
54 ENCLOSED BY '"'
55 LINES TERMINATED BY "\n"
56 IGNORE 1 ROWS;
57
```

Output

#	Time	Action	Message
132	20:06:30	CREATE TABLE IF NOT EXISTS products (id INT PRIMARY KEY, product_name VARCHAR(255), price ...	0 row(s) affected
133	20:06:34	LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N3.Ex.2__products.csv" INTO TAB...	Error Code: 1366. Incorrect decimal value: '\$161.11' for column 'price' at row 1

Lo intentamos de nuevo cambiando con **SET** este símbolo del \$.

```
48 LOAD DATA
49 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N3.Ex.2__products.csv"
50 INTO TABLE products
51 FIELDS TERMINATED BY ","
52 ENCLOSED BY '"'
53 LINES TERMINATED BY "\n"
54 IGNORE 1 ROWS
55 SET price = CAST(REPLACE(TRIM(@price), '$', '') AS DECIMAL(10,2));
56
```

Output

#	Time	Action	Message
134	20:07:43	LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N3.Ex.2__products.csv" INTO TAB...	Error Code: 1366. Incorrect decimal value: '\$161.11' for column 'price' at row 1
135	20:08:45	LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N3.Ex.2__products.csv" INTO TAB...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Ya tenemos los datos cargados dentro de nuestra nueva tabla.

```
14 SELECT * FROM product_transactions.products;
```

Result Grid

id	product_name	price	colour	weight	warehouse_id
1	Direwolf Stannis	161.11	#7c7c7c	1.00	WH-4
2	Tarly Stark	9.24	#919191	2.00	WH-3
3	duel tourney Lannister	171.13	#d8d8d8	1.50	WH-2
4	warden south duel	71.89	#111111	3.00	WH-1
5	skywalker ewok	171.22	#bdbdbd	3.20	WH-0
6	dooku solo	136.60	#c4c4c4	0.80	WH--1
7	north of Casterly	63.33	#b7b7b7	0.60	WH--2

products 2 x

Output

#	Time	Action	Message
129	20:03:55	SELECT * FROM product_transactions.products	100 row(s) returned
130	20:04:06	SELECT * FROM product_transactions.products	100 row(s) returned

3

Ejercicio 2

Crea una tabla para unir los datos del archivo new products.csv con la base de datos creada, teniendo en cuenta que dispones de los product_ids de la tabla de transacciones. Genera la siguiente consulta:

👉 Necesitamos saber el número de veces que se ha vendido cada producto.

Seleccionamos el id y el nombre del producto de la tabla *products*. Hacemos un recuento del número del número de id y agrupados. Obtenemos un listado con el resultado total de veces que se ha vendido cada producto.

```
54 • SELECT products.id, products.product_name,
55     COUNT(transactions.product_ids) AS num_sold
56 FROM transactions
57 JOIN products
58 ON transactions.product_ids = products.id
59 GROUP BY products.id, products.product_name
60 ORDER BY products.id ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

id	product_name	num_sold
1	Direwolf Stannis	990
2	Tarly Stark	1034
3	duel tourney Lannister	1027
4	warden south duel	960
5	skywalker evok	1030

Result 42 x

Output

Action Output

#	Time	Action	Message
✓	103 20:10:36	SELECT products.id, products.product_name, COUNT(*) AS num_sold FROM transactions JOIN products ON ...	100 row(s) returned
✓	104 20:16:50	SELECT products.id, products.product_name, COUNT(transactions product_ids) AS num_sold FROM transact...	100 row(s) returned
✓	105 20:19:22	SELECT products.id, products.product_name, COUNT(transactions product_ids) AS num_sold FROM transact...	100 row(s) returned

Vamos a comprobarlo:

¿cuántas veces aparece el producto con ID 1 en la tabla *transactions*?
990

```
12 • SELECT product_ids
13 FROM product_transactions.transactions
14 where product_ids = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

product_ids
1, 2, 7, 72
1, 95, 44, 26
1, 35, 32
1, 67, 20
1

transactions 10 x

Output

Action Output

#	Time	Action	Message
✓	122 10:15:24	SELECT product_ids FROM product_transactions.transactions where product_ids = 1	990 row(s) returned

3

Ejercicio 2

Crea una tabla para unir los datos del archivo new products.csv con la base de datos creada, teniendo en cuenta que dispones de los product_ids de la tabla de transacciones. Genera la siguiente consulta:

👉 Necesitamos saber el número de veces que se ha vendido cada producto.

Lo primero que vamos a hacer es desglosar la columna *product_id* de la tabla *transactions* para que cada producto vendido aparezca como un registro. Esto significa que se multiplicarán las transacciones por el número de ids de cada columna.

Lo hacemos con una tabla **Json**. Con la cláusula **CONCAT** convertimos los elementos de la columna en una lista.

Con **INT PATH**, sacamos cada valor como un número.

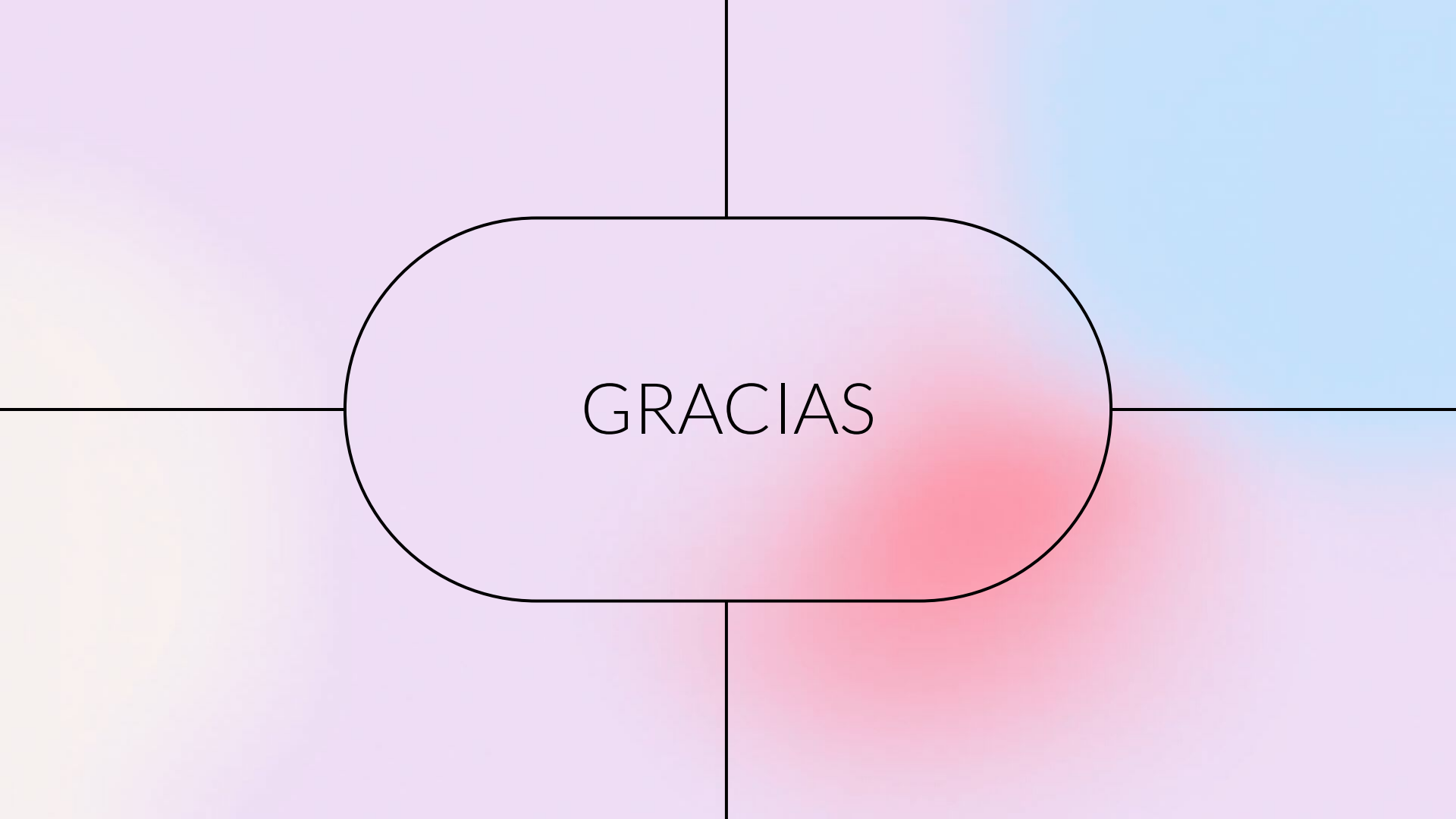
```
5 • SELECT
6     transactions.id,
7     list_prod.product_id
8 FROM transactions
9 JOIN JSON_TABLE(
10     CONCAT('[', transactions.product_ids, ']'),
11     '$[*]' COLUMNS (
12         product_id INT PATH '$'
13     )
14 ) AS list_prod;
```

Result Grid	Filter Rows:	Exports:	Wrap Cell Contents	Fetch rows:
id	product_id			
00043A49-2949-494B-ASDO-A5BAE38B19DD	16			
00043A49-2949-494B-ASDO-A5BAE38B19DD	26			
00043A49-2949-494B-ASDO-A5BAE38B19DD	97			
00043A49-2949-494B-ASDO-A5BAE38B19DD	87			
00045D6B-ED2E-4F2F-8186-CEE074D875D0	30			

Result 10 x

Output

#	Time	Action	Message
35	13:36:59	SELECT transactions.id, desglosed_prod product_id FROM transactions JOIN JSON_TABLE(CONC...	253388 row(s) returned
36	13:37:16	SELECT transactions.id, desglosed_prod product_id FROM transactions JOIN JSON_TABLE(CONC...	Error Code: 1064. You have an error in
37	13:38:32	SELECT transactions.id, list_prod.product_id FROM transactions JOIN JSON_TABLE(CONCAT(T, tr...	253388 row(s) returned



GRACIAS